



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Laboratorio 3

Sistemas de información para la gestión

RUT: 21.824.858-6

DIEGO ALONSO ZUÑIGA CORTEZ

Semilla S = 858 | Umbral de monto U = \$38.000 | Plazo máximo D = 21 días

Repositorio: <https://github.com/Diego2704-rgb/ICN292-LAB3-Zuniga-Diego>

Resumen ejecutivo

Durante este laboratorio construí en n8n un producto mínimo viable (MVP) para el triage de las solicitudes de devolución del canal e-commerce de AndesHogar SpA, el mismo proceso que modelé en BPMN en el Laboratorio 2, donde el cuello de botella estaba en decidir para cada solicitud si se aprueba, si pasa a revisión en bodega o si se rechaza. A partir de los últimos 3 dígitos de mi rut antes del verificador, 858, obtuve el umbral de monto $U = 30.000 + 1.000 \times (858 \bmod 50) = 38.000$ pesos y el plazo máximo $D = 7 + 7 \times (858 \bmod 4) = 21$ días. Con esto armé tres workflows: el de triage, que recibe cada solicitud por un webhook, la clasifica con un Switch cuyas reglas siguen el orden de la política, escribe ruta y motivo, consulta la UF en mindicador.cl, registra todo en una Data Table, notifica por correo y responde; el emisor, que envía las 15 solicitudes por POST a la URL de producción; y el de resumen, que corre todos los días a las 08:00, consolida el día anterior con Summarize y envía un solo correo.

Las 15 solicitudes quedaron registradas con su motivo y los valores de U y D aplicados: 2 aprobadas automáticamente (\$29.980), 9 a revisión (\$1.020.890) y 4 rechazadas (\$200.950), con un total de \$1.251.820 y una tasa de aprobación automática de solo 13,3%. Las tres solicitudes de control cayeron donde debían. En la parte C probé tres entradas inválidas, los dos casos de frontera (monto = U y días = D, ambos aprobados por la regla 6), el error de tipo y los dos casos que el flujo no puede resolver. Las evidencias son las ejecuciones 5 a 19 del triage, la 4 del emisor, la 20 del resumen, las 21 a 28 de la parte C, los tres workflows exportados y el repositorio <https://github.com/Diego2704-rgb/ICN292-LAB3-Zuniga-Diego>.

Recomendación. Recomiendo que el MVP sí pase a ser el sistema definitivo, pero no tal como está, sino después de cerrar dos brechas que quedaron demostradas en la parte C. A favor está que clasificó correctamente las 15 solicitudes y los dos casos de frontera, que cada solicitud se resolvió en 1 a 3 segundos frente a los 3 días hábiles actuales de primera respuesta (la política promete 24 horas) y que cada decisión queda registrada con su motivo, por lo que es auditable. En contra, el flujo no valida el formato de los datos ni el catálogo: un monto escrito en palabras hace fallar el Switch sin responder nada útil al cliente, y un SKU inexistente se aprueba como si fuera válido. Además, no tiene Error Workflow, por lo que si el resumen programado falla nadie se entera. Antes de dejarlo como definitivo agregaría un paso de validación previo al Switch (monto y días numéricos, SKU presente en el catálogo) que mande esos casos a DATOS_INVALIDOS, y un Error Workflow que avise por correo. Por último, con $U = 38.000$ seis de las nueve revisiones ocurren solo por monto, por lo que la gerencia debería evaluar si el umbral es el adecuado, ya que el ahorro del MVP depende de cuántas solicitudes resuelve sin intervención humana.

Parte A: Flujo de triage

Para esta parte lo primero fue traducir las reglas de negocio a un workflow de n8n sin escribir código. El workflow quedó con 12 nodos y ninguno es de tipo Code. Comienza con un nodo Webhook que recibe las solicitudes por POST en el path andeshogar-devoluciones y que está configurado para responder recién al final, mediante el nodo Respond to Webhook. Luego viene el Switch en modo Rules, que es el corazón del triage: cada condición quedó en su propia regla y en el mismo orden de la política, de modo que la primera que se cumple fija la ruta y detiene la evaluación. Para comparar números usé `.toNumber()` dentro de cada condición, ya que todos los campos llegan como texto, tal como los enviaría un formulario web. El Fallback Output quedó como la rama de aprobación y la opción Send data to all matching outputs quedó desmarcada, para que una solicitud no pueda salir por dos ramas al mismo tiempo. El Switch quedó así:

#	Condición	Detalle	Salida
1	String · is empty	{{ \$json.body.id_solicitud }}	INVALIDO_CAMPO
2	Number · ≤ 0	{{ \$json.body.monto.toNumber() }}	INVALIDO_MONTO
3	Number · > 21	{{ \$json.body.dias_desde_compra.toNumber() }}	RECHAZO_PLAZO
4	String · equal to	estado_producto = danado_por_uso	RECHAZO_ESTADO
5	Number · > 38000	{{ \$json.body.monto.toNumber() }}	REVISION_MONTO
6	String · equal to	estado_producto = con_fallas	REVISION_FALLA
—	Fallback Output	—	APROBACION

Tabla 1: Reglas del Switch con los valores de mi semilla ($U = 38.000$, $D = 21$).

Las siete salidas llegan a cuatro nodos Edit Fields, uno por ruta, y cada uno escribe los campos ruta y motivo. Como a un mismo Edit Fields llegan dos causales distintas, para que el motivo diga cuál de las dos se cumplió usé la expresión `$prevNode.outputIndex`, que indica por qué salida del Switch llegó la solicitud; así, por ejemplo, el nodo de RECHAZO escribe “RECHAZO_PLAZO: dias_desde_compra = 25 $>$ D = 21” si vino por la salida 2, o “RECHAZO_ESTADO: estado_producto = danado_por_uso” si vino por la salida 3, sin tener que repetir la lógica del Switch en otro lugar.

Después de la ruta, el flujo consulta el servicio público <https://mindicador.cl/api/uf>, que no pide credenciales, y consume el campo `serie[0].valor`, que corresponde al valor más reciente de la UF (el 21 de septiembre de 2026 fue \$40.983,58). Un detalle que tuve que resolver es que el nodo HTTP Request reemplaza los datos que recibe por la respuesta del servicio, por lo que se perdían la ruta y el motivo; para evitarlo agregué un nodo Merge en modo combinar por posición, que junta en un solo ítem la salida de la rama con la respuesta de la UF. Con eso, el nodo Construir registro arma la fila con todos los campos de la regla 7 y algunos extra: fecha, fecha y hora, id_solicitud, sku, unidades, monto, valor de la UF, monto en UF, días desde la compra, estado, correo del cliente, ruta, motivo, $U = 38.000$ y $D = 21$. Esa fila se guarda en una Data Table de n8n llamada registro_devoluciones, luego se

notifica al cliente con un nodo de Gmail y finalmente el nodo Respond to Webhook devuelve el registro completo a quien envió la solicitud.

Cabe destacar que la notificación se envía desde un correo secundario que conecté para esto y llega a mi propio buzón, indicando en el asunto el destinatario real (por ejemplo “[MVP - para cliente1@example.cl]”). Tomé esta decisión porque las direcciones de los datos de prueba no son buzones que yo controle, y en un MVP no corresponde enviar correos a terceros desconocidos; en el sistema definitivo bastaría con cambiar el destinatario por el campo email_cliente.

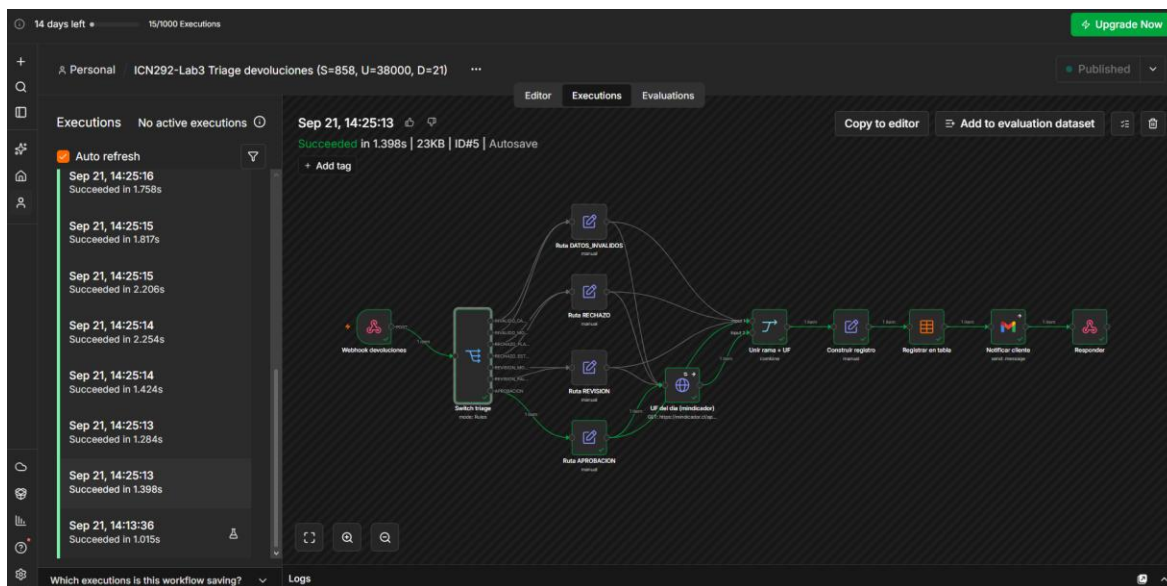


Imagen 1: Vista general del workflow de triage en la ejecución 5 (DEV-1115), con el camino seguido marcado en verde.

Para enviar las solicitudes construí un segundo workflow, el emisor, que parte con un Trigger manually, luego un Edit Fields que contiene las 15 solicitudes en JSON, un Split Out que las separa en 15 ítems y un HTTP Request que hace un POST por cada una hacia la URL de producción del webhook, con una pausa de medio segundo entre cada envío. Respecto a las dos URL, según la documentación oficial de n8n la URL de prueba (/webhook-test/) se registra solo cuando uno pulsa “Listen for test event” o ejecuta el workflow desde el editor, atiende una llamada y muestra los datos directamente en el canvas; en cambio la URL de producción (/webhook/) se registra cuando el workflow se publica y sus datos no aparecen en el canvas, sino en la pestaña Executions. Para las 15 solicitudes usé la URL de producción, porque la de prueba atiende una sola llamada por escucha y el emisor necesitaba enviarlas seguidas; esto se ve en las capturas, donde el campo executionMode dice production y la webhookUrl contiene /webhook/. Las pruebas de la parte C, en cambio, las hice como ejecuciones manuales de prueba, que aparecen en la lista marcadas con el ícono de matraz.

El emisor se ejecutó una vez (ejecución 4) y generó las ejecuciones 5 a 19 del triage, todas exitosas. Los resultados registrados fueron los siguientes:

Solicitud	Monto (\$)	Días	Estado	Ruta	Motivo registrado
DEV-1115	19.990	3	sin_abrir	APROBACION	dias $3 \leq 21$; monto $19.990 \leq 38.000$
DEV-1116	69.980	5	abierto_sin_uso	REVISION	REVISION_MONTO: $69.980 > U$
DEV-1117	24.990	12	con_fallas	REVISION	REVISION_FALLA: con_fallas
DEV-1118	79.990	4	sin_abrir	REVISION	REVISION_MONTO: $79.990 > U$
DEV-1119	45.990	25	danado_por_uso	RECHAZO	RECHAZO_PLAZO: $25 > D$
DEV-1120	89.990	2	con_fallas	REVISION	REVISION_MONTO: $89.990 > U$
DEV-1121	289.990	9	sin_abrir	REVISION	REVISION_MONTO: $289.990 > U$
DEV-1122	45.990	2	sin_abrir	REVISION	REVISION_MONTO: $45.990 > U$
DEV-1123	39.980	15	danado_por_uso	RECHAZO	RECHAZO_ESTADO: danado_por_uso
DEV-1124	49.980	1	abierto_sin_uso	REVISION	REVISION_MONTO: $49.980 > U$
DEV-1125	34.990	22	sin_abrir	RECHAZO	RECHAZO_PLAZO: $22 > D$
DEV-1126	79.990	8	con_fallas	REVISION	REVISION_MONTO: $79.990 > U$
DEV-C01	9.990	1	sin_abrir	APROBACION	dias $1 \leq 21$; monto $9.990 \leq 38.000$
DEV-C02	289.990	2	sin_abrir	REVISION	REVISION_MONTO: $289.990 > U$
DEV-C03	79.990	29	abierto_sin_uso	RECHAZO	RECHAZO_PLAZO: $29 > D$

Tabla 2: Resultado del triage de las 15 solicitudes, motivo abreviado ($U = 38.000$, $D = 21$).

En esta tabla hay dos casos que sirven para comprobar la precedencia de la política. DEV-1119 tiene 25 días y además está dañado por uso, y quedó como RECHAZO_PLAZO porque la regla de plazo se evalúa antes que la de estado; del mismo modo, DEV-1120 tiene fallas y un monto de \$89.990, y quedó como REVISION_MONTO porque la regla del monto va antes que la de fallas. Las tres solicitudes de control, cuya ruta no depende de la semilla, quedaron como corresponde: DEV-C01 aprobada, DEV-C02 a revisión por monto y DEV-C03 rechazada por plazo. Las siguientes capturas muestran el panel del Switch en las ejecuciones 5 y 9, con la solicitud a la izquierda y la salida tomada a la derecha.

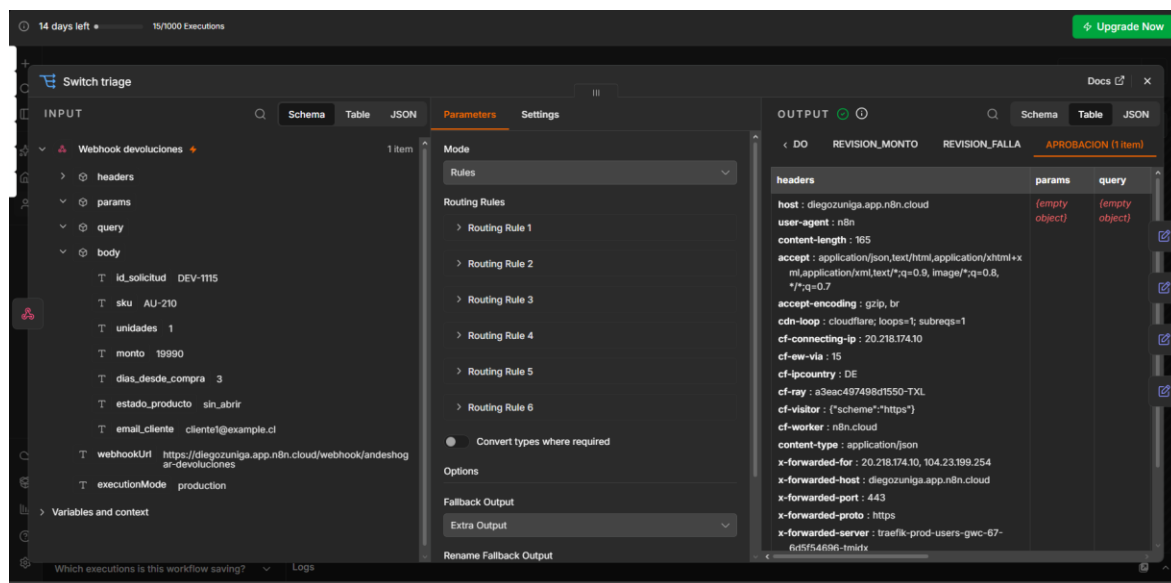


Imagen 2: Ejecución 5 (producción): DEV-1115, \$19.990, 3 días, sin abrir → salida APROBACION (fallback).

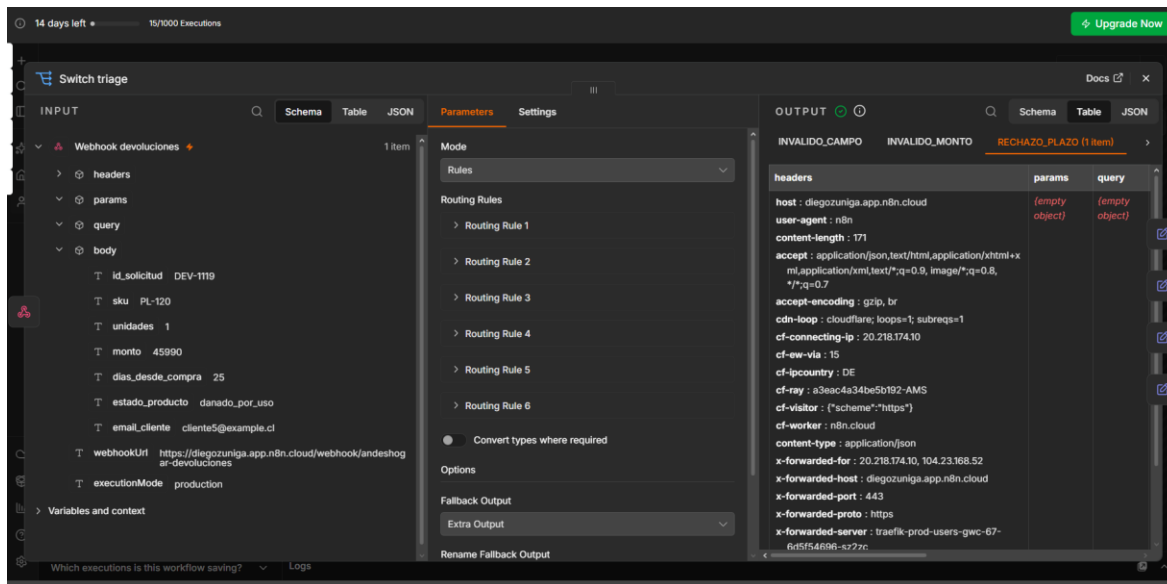


Imagen 3: Ejecución 9 (producción): DEV-1119, 25 días y danado_por_uso → RECHAZO_PLAZO, la regla de plazo gana por precedencia.

Parte B: Flujo programado de resumen

Para el resumen construí un tercer workflow que parte con un Schedule Trigger configurado para ejecutarse todos los días a las 08:00, con la zona horaria de Santiago. Elegí esta frecuencia porque el triage funciona de forma continua y lo que necesita la gerencia es una foto del día completo; si el resumen se ejecutara a las 23:59 del mismo día, las solicitudes que llegan en ese último minuto quedarían fuera y ya no se contarían nunca, mientras que ejecutándolo a las 08:00 sobre el día anterior el día ya está cerrado y el correo llega justo al inicio de la jornada, que es cuando se puede actuar sobre las solicitudes en revisión. Además agregué un Trigger manually que resume el día actual, solo para poder probarlo y dejar evidencia sin esperar al día siguiente.

Después del trigger, el flujo lee de la Data Table las filas cuya fecha coincide con el día a resumir y las consolida con dos nodos Summarize: el primero agrupa por ruta y calcula la cantidad de solicitudes y la suma del monto, y el segundo suma esos resultados para obtener los totales del día. Con eso un Edit Fields arma el mensaje, incluida la tasa de aprobación automática, que calculo como solicitudes aprobadas dividido por el total de solicitudes del día, y un nodo de Gmail lo envía como un solo correo. La ejecución 20, hecha con las 15 solicitudes ya registradas, envió el siguiente mensaje:

Ruta	Solicitudes	Monto total (\$)
APROBACION	2	29.980
REVISION	9	1.020.890
RECHAZO	4	200.950
Total del día	15	1.251.820
Tasa de aprobación automática	2 / 15	13,3 %

Tabla 3: Contenido del correo de resumen del 21-09-2026 (ejecución 20).

Respecto a qué ocurre un día sin solicitudes, lo probé antes de enviar las 15, con la tabla vacía, y en esa primera prueba el flujo falló: la Data Table no devolvió filas, entonces el nodo Summarize se detuvo con el error “The field id_solicitud does not exist in any items” y el correo nunca se habría enviado. Esto es grave en un flujo programado, porque para la gerencia no llegar el correo se ve igual que no haber recibido nada, cuando en realidad el sistema está caído. Lo corregí haciendo que la lectura de la tabla siempre entregue un resultado (aunque sea vacío) y activando en ambos Summarize la opción de continuar si no encuentra el campo. Con esto, un día sin solicitudes genera igualmente el correo, que informa 0 solicitudes, monto \$0, la línea “Sin solicitudes registradas” y la tasa como “no aplica (0 solicitudes)”, ya que dividir por cero no tiene sentido.

Parte C: Errores y casos borde

Para esta parte envié varias solicitudes construidas a propósito. Para no ensuciar el registro oficial, que es el que lee el resumen, durante estas pruebas el triage escribió en una tabla aparte llamada registro_pruebas_desarrollo, y al terminar lo dejé apuntando de nuevo a la tabla oficial.

Tres entradas inválidas

Ejec.	Carga enviada	Comportamiento esperado	Comportamiento observado
21	id_solicitud: "" (resto válido: AU-210, \$19.990, 3 días, sin_abrir)	DATOS_INVALIDOS por la regla 1	Salida INVALIDO_CAMPO; registrado con motivo “INVALIDO_CAMPO: falta id_solicitud”
22	id DEV-C-INV2, monto: "-24990", 4 días, sin_abrir	DATOS_INVALIDOS por la regla 2	Salida INVALIDO_MONTO; motivo “monto = -24990 no es mayor que cero”
23	id DEV-C-INV3, \$34.990, días_desde_compra: "tres"	Que falle un nodo, ya que el texto no se puede convertir a número	El Switch falla con “cannot convert to number”; la ejecución queda en Error y no se registra nada

Tabla 4: Entradas inválidas: carga, comportamiento esperado y observado.

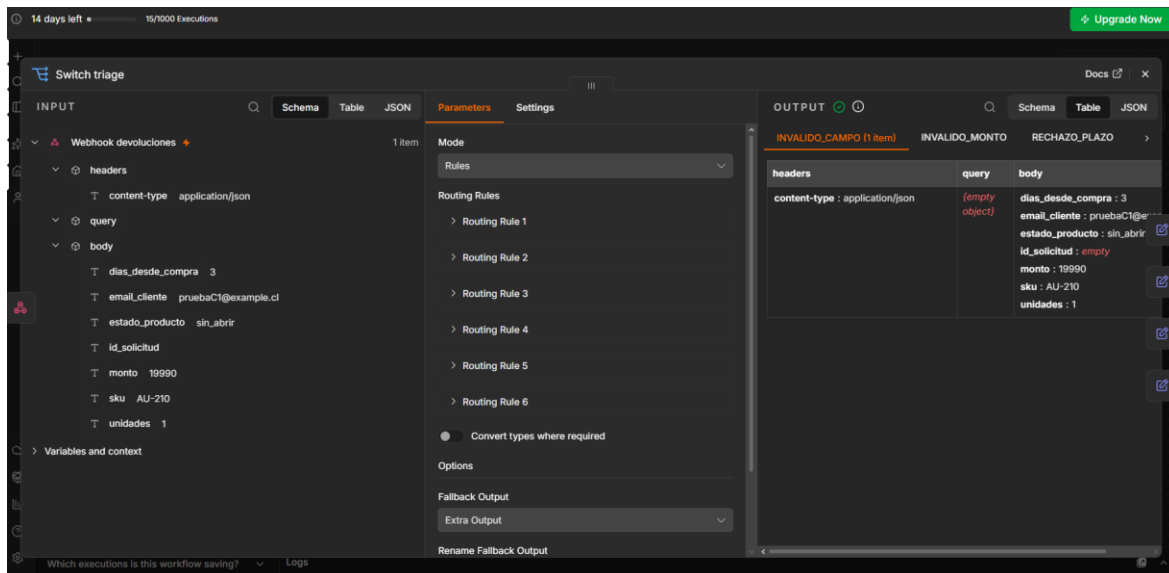


Imagen 4: Ejecución 21: id_solicitud vacío → salida INVALIDO_CAMPO.

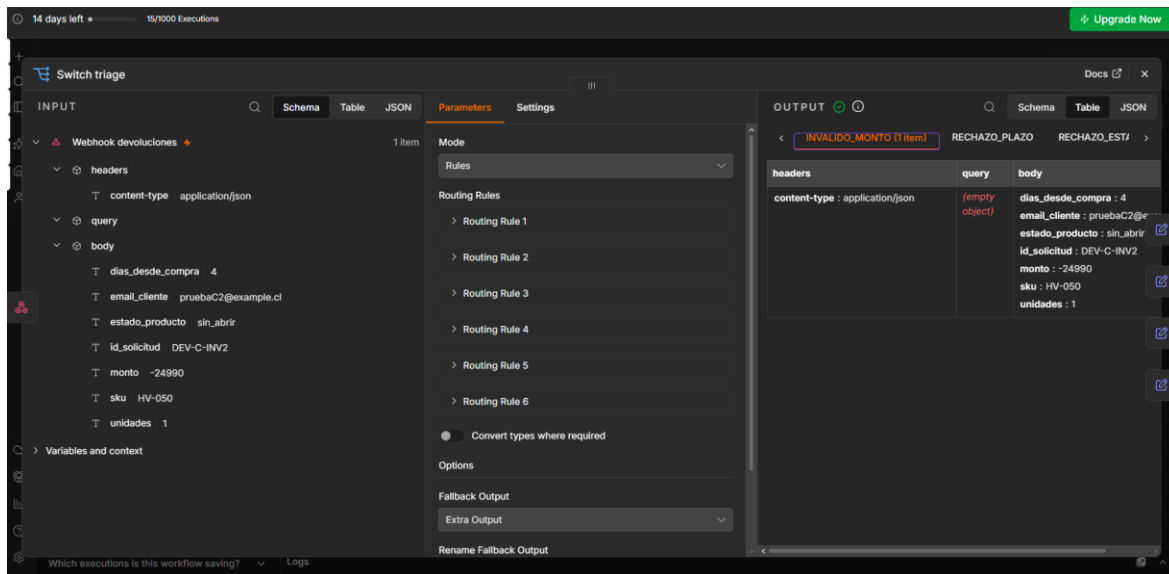


Imagen 5: Ejecución 22: monto negativo → salida INVALIDO_MONTO.

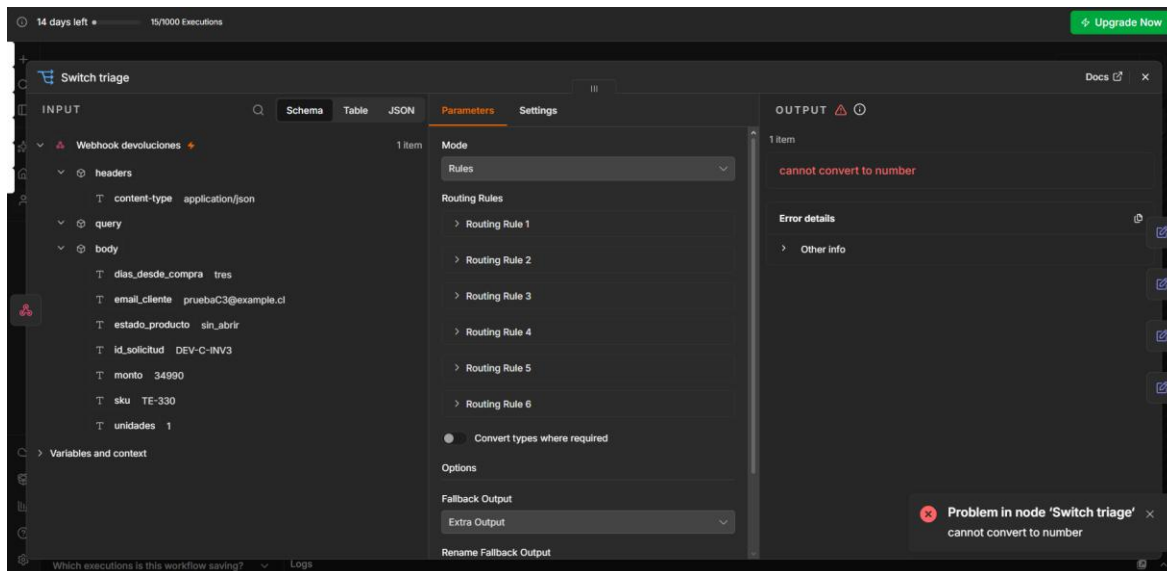


Imagen 6: Ejecución 23 (fallida): días escritos en palabras hacen fallar el Switch.

Los tres valores de On Error

n8n permite definir en cada nodo qué hacer si falla, con tres valores posibles. Stop Workflow, que es el valor por defecto, detiene toda la ejecución y la marca como error. Continue deja que el flujo siga por la salida normal, llevando el mensaje de error como dato. Continue (using error output) le agrega al nodo una segunda salida por donde se va el ítem que falló, para poder tratarlo en una rama distinta. Lo que elegí en cada nodo fue lo siguiente:

Nodo	On Error	Razón
Switch triage	Stop Workflow	Si una solicitud no se puede evaluar no debe clasificarse ni registrarse con una ruta que no le corresponde; es preferible que falle a la vista.
Edit Fields, Merge y Construir registro	Stop Workflow	Son transformaciones internas; si fallan, el registro saldría incompleto.
Registrar en tabla	Stop Workflow	Por la regla 7, una decisión sin registro no es auditable, así que no debe notificarse ni responderse algo que no quedó guardado.
UF del día (mindicador)	Continue + 3 intentos	La UF es informativa y no participa en la decisión; si el servicio externo no responde, la solicitud se registra igual con la UF vacía.
Notificar cliente (Gmail)	Continue	La decisión ya quedó registrada; un correo que falla no debería impedir responder. En la primera prueba, sin credencial, el flujo siguió y el error quedó guardado en la salida del nodo.
Responder	Stop Workflow	Es el último paso; si falla, debe verse como error.

Tabla 5: Valor de On Error elegido en cada nodo del triage.

No usé Continue (using error output), pero sería la opción correcta en una versión definitiva para el nodo de Gmail, ya que permitiría mandar las notificaciones fallidas a una rama que las deje marcadas como pendientes de reenvío, en vez de solo seguir de largo.

Error Workflow

Un Error Workflow es un workflow aparte que comienza con un nodo Error Trigger y que se asigna en la configuración de otro workflow. Cada vez que una ejecución de producción de ese otro workflow falla, n8n ejecuta automáticamente el Error Workflow, entregándole qué workflow falló, en qué nodo y con qué mensaje, para que por ejemplo mande un correo de alerta. Un flujo programado sin Error Workflow deja de funcionar en silencio porque nadie lo está mirando: se ejecuta solo a las 08:00, y si falla, el error queda guardado en la lista de ejecuciones pero nadie recibe aviso. En este caso el síntoma sería simplemente que el correo de resumen no llega, lo que se puede confundir con un día sin novedades. Esto no es teórico, ya que en la parte B el Summarize falló justamente así con la tabla vacía, y sin haberlo probado a mano no me habría enterado.

Límites

Como las 15 solicitudes entregadas no incluyen casos de frontera, construí dos adicionales para mi semilla, una con monto exactamente igual a U y otra con días exactamente iguales a D:

Ejec.	Solicitud	Monto	Días	Estado	Ruta obtenida
24	DEV-LIM-U	38.000 (= U)	5	sin_abrir	APROBACION: "monto = 38000 <= U = 38000"
25	DEV-LIM-D	19.990	21 (= D)	sin_abrir	APROBACION: "dias = 21 <= D = 21"

Tabla 6: Casos de frontera contruidos con la semilla 858.

Ambos quedaron dentro de lo aceptado, tal como dice la regla 6. Esto ocurre porque las condiciones del Switch usan “mayor que” estricto (monto > 38.000 y días > 21), por lo que un valor exactamente igual al límite no activa la regla y la solicitud sigue hasta el fallback de aprobación.

El error de tipo, a propósito

Para esta prueba cambié el tipo de la condición del monto (regla 5) de Number a String y envié DEV-C01, que tiene un monto de \$9.990. El resultado fue que el Switch falló con el mensaje “Comparison type expects a string but both fields are a number” (ejecución 28), es decir, no obtuve ninguna ruta.

Explicación: la validación estricta de tipos de n8n detectó que se estaba comparando un número como si fuera texto y detuvo la ejecución. Sin esa protección la comparación sería alfabética, donde “9990” es mayor que “38000” porque el carácter 9 va después del 3, y DEV-C01 habría terminado equivocadamente en REVISION.

Una vez hecha la prueba revertí la condición a Number, lo que se puede comprobar en el workflow exportado.

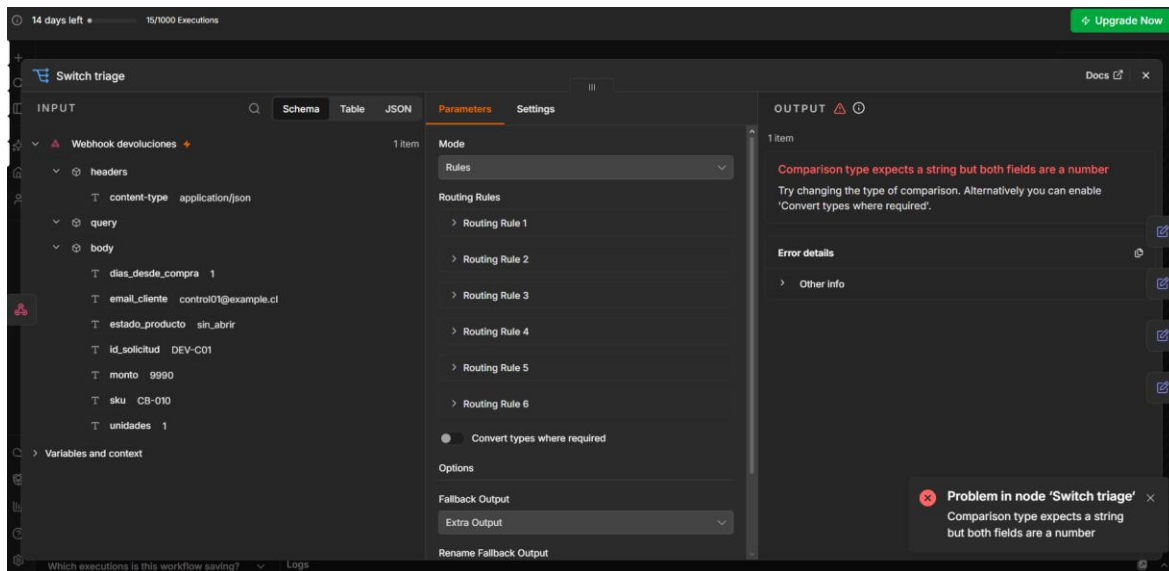


Imagen 7: Ejecución 28: con la regla del monto en String, DEV-C01 hace fallar el Switch.

Los dos casos que el flujo no puede resolver

Monto "veinte mil" (ejecución 26). El flujo asume que el monto viene escrito como número, pero `.toNumber()` no sabe interpretar palabras, así que el Switch falla antes de clasificar y la solicitud no queda registrada ni recibe una respuesta útil. Para detectarlo, el flujo necesitaría una validación previa al Switch que revise que monto y días contengan solo dígitos y que, si no, mande la solicitud a DATOS_INVALIDOS con un motivo, en vez de fallar.

SKU "ZZ-999" (ejecución 27). Las reglas solo miran el monto, los días y el estado, y ninguna revisa que el SKU exista, así que la solicitud pasó la validación y quedó registrada como APROBACION, igual que una válida. Para detectarlo, el flujo necesitaría buscar el SKU en el catálogo (por ejemplo, en otra Data Table) y enviarlo a DATOS_INVALIDOS si no existe, y de paso comprobar que el monto coincida con el precio por las unidades.

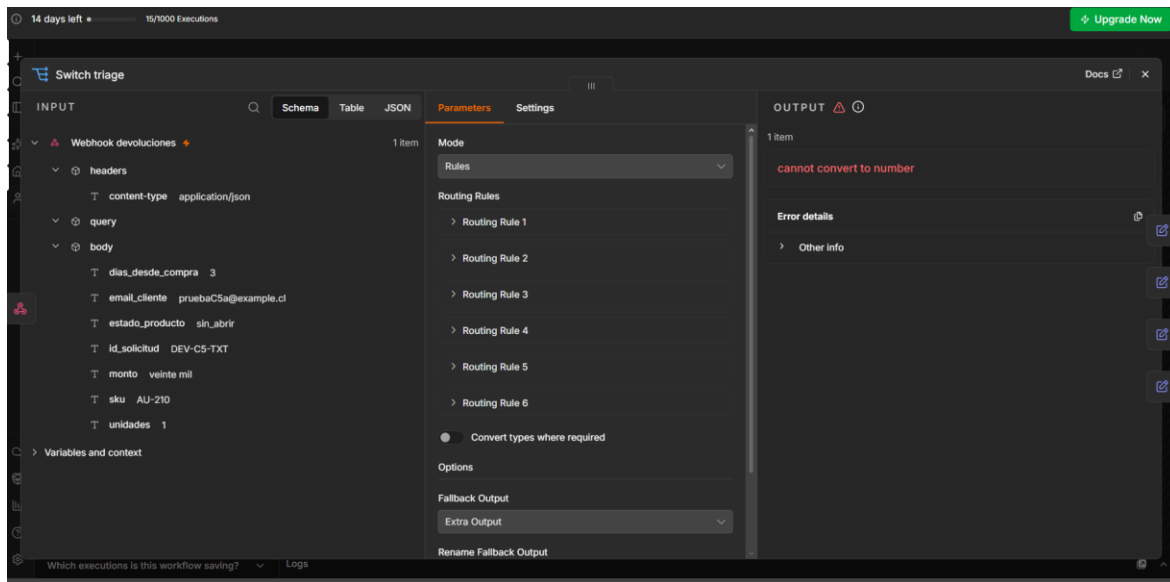


Imagen 8: Ejecución 26: monto "veinte mil" → el Switch falla con "cannot convert to number".